Computer Graphics: Lecture 18

Advanced Texturing

Slide Deck © Grant Williams, 2026, License: CC-BY-SA 4.0

Welcome back!

Last time we learned all about cube topologies.

They were surprisingly involved, and we learned two in particular:

- UV-spheres
- Cube-spheres

[How do they differ, and does anyone recall the secret 3rd option?]

We also learned how to texture map them. UV-spheres straightforwardly wrap a texture around them cylindrically.

Cube-spheres *could* use the same trick, but there's a better way…

Welcome back! (2)

We saw that there exists a special kind of texture that is designed to be sampled with a vector, as if the texture is on the surface of a cube, and the sample is a vector pointing outward from the center of the cube.

It's called a **cube-map**, and we learned how to load and to use them.

Cube-maps allow textures to be sampled with a *direction* instead of a point.

They can also unlock two very cool techniques:
1. Sky boxes. Very important to 3D games in particular.
2. Environment maps: cool reflections which are cheap to compute.

This time

So what's up with the title of this lecture, *Advanced Texturing*? Those cube maps seemed pretty advanced to me!

Well, there have been a lot of important aspects of tetures we've been ignoring so far namely:

- How they even work. Like, how does the GPU does sampling?
- And how does that sampling work with perspective?
- And what happens if the texture is too far or too close?
- And what happens if the texture is at an angle?

It might not have occurred to you that these would be problems, but...

How do textures even work?

Let's start by reminding ourselves how textures work.

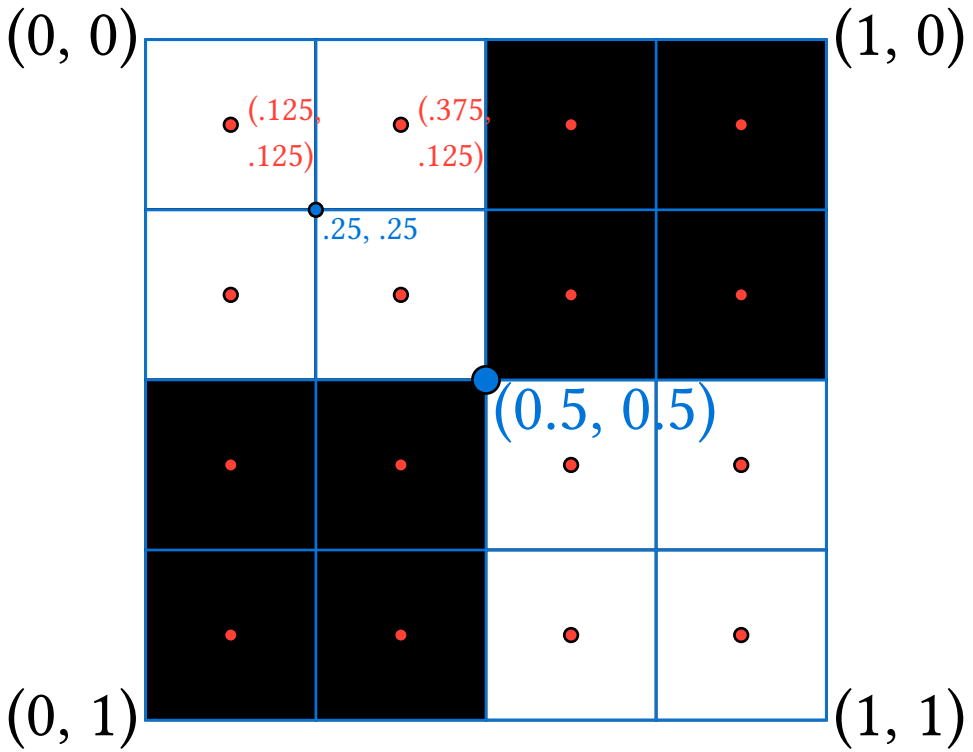
For now, we're going to restrict our discussion to 2D flat textures.

2D (non-cube) textures are sampled with 2-D coordinates in UV space.

That means, `0, 0` is the top left corner of the texture, and `1, 1` is the bottom right corner.

Let's make a really simple texture so we can plot the UVs…

A very simple tile texture



A 4-by-4 texel texture. Notice that the centers of the texels are offset by half a texel in UV space.

Bilinear filtering

We've seen that we sample textures using fractions. What happens when one of them doesn't correspond to the center of the pixel?

For example, what if we sample (0.5, 0.5) in the previous image?

We can see that it's in the center, which means it's actually between 4 texels. But two of those texels are black, and two are white. What is the color that we get back?

Answer: it depends on our sampling mode.

Bilinear filtering (2)

In `'nearest'` mode, we perform nearest neighbor sampling. If we were closer to a black texel we would get black, and if we were closer to a white texel we would get white.

In this case, we are equally close to either color, so it's up to the implementation which one we get.

But in `'linear'` mode, we perform linear filtering in two dimensions. This is called **bilinear filtering**...

Bilinear filtering (3)



Left: a 2-by-2 image with perfect fidelity, blown up.
Right: a 2-by-2 image smoothly filtered bilinearly

But that looks horrible!

In this case, the texture happens to be perfect for nearest filtering, which is why it's an option.

However, keep in mind that most textures are meant to be in the background, and having sharply defined pixels makes them stand out…

It looks better in practice

Sometimes we like the light blurring, especially for background elements and natural textures. Example: this image from learnopengl.com.

Also keep in mind that stretching a 2-by-2 pixel texture over half the screen is a pretty extreme case. It's usually not as noticeable.

So how does the math behind this work?

Bilinear filtering (4)

When we sample a texture, we also look up the values of the 4 closest texels that surround the point we sampled.

We compute two mean V values: one between the top two points and one between the bottom two points.

Those means are weighted by how close we are to each point. If we are 30% from the right side, we compute `(30% * left_side + 70% * right_side)`. We do this *twice*, once for the two texels above our sample, and once for the two texels below our sample.

Bilinear filtering (5)

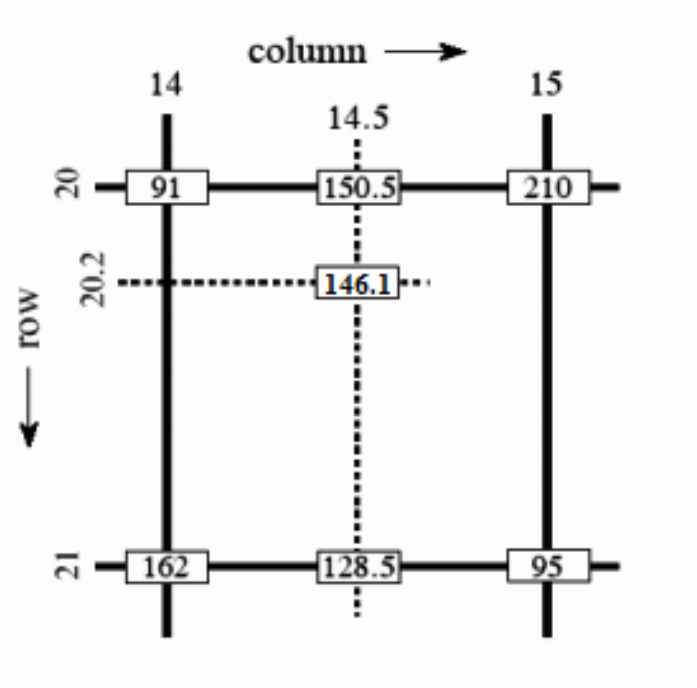
Then, we average those two averages based on how close the V coordinate is from them.

So if we're 10% away from the top two texels in the V dimension, we would compute `(100% - 10%) * top + 10% * bottom`.

This final average would be our result.

Here's an illustration…

Bilinear filtering in grayscale



Suppose we sample `(14.5, 20.2)`. First, this is halfway between `91` and `210` on top, so the top average is `150.5`. Likewise, the bottom average is exactly between `162` and `95`, resulting in `128.5`. We are 20% away from the top sample, so we get `150.5 * 80% + 128.5 * 20% = 146.1`

Image by Jayson Kostelyk, Vizerai; Public Domain

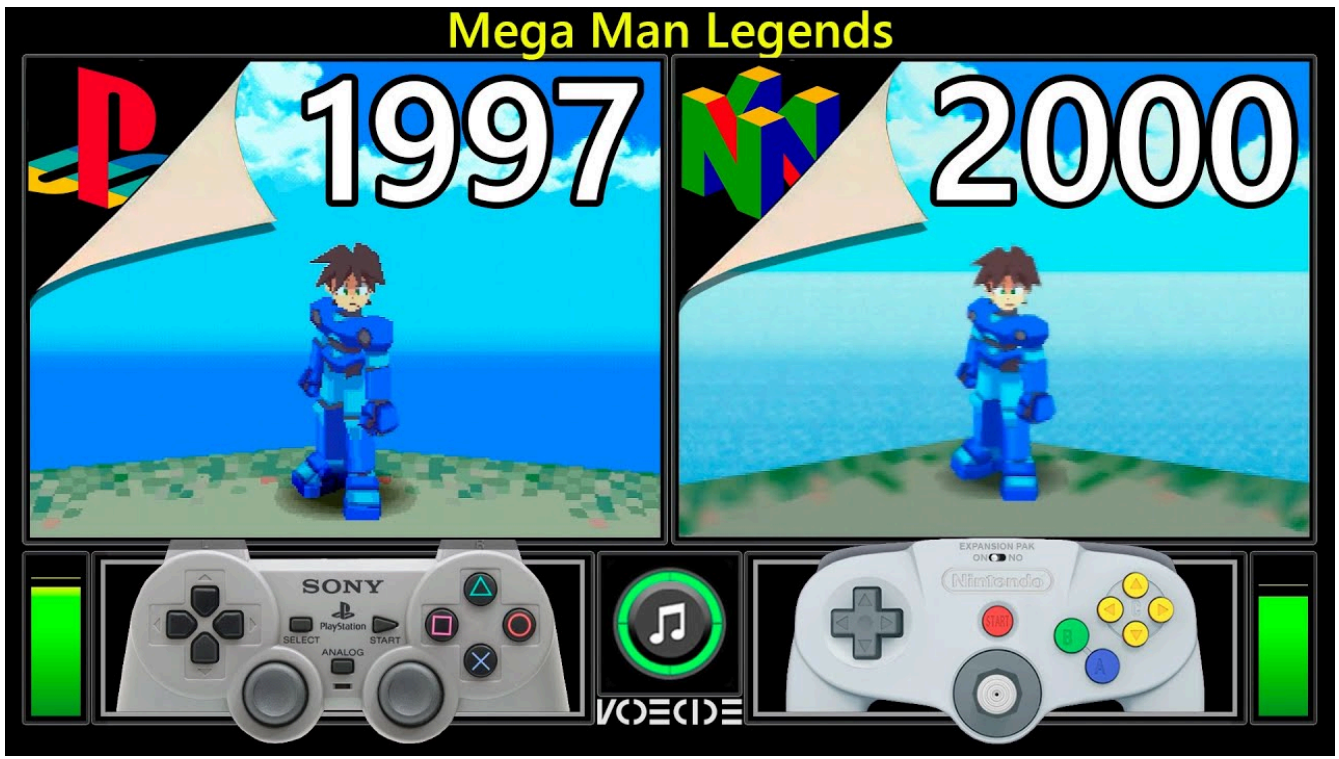
Bilinear filtering in color

To do this in RGB space, we just apply the same technique to the R, G, and B channels separately.

Either way, we get the ability to smoothly sample the texture.

In the early days, the Nintendo 64 supported this kind of texture filtering, while the Playstation didn't. It led to the distinct difference between the two consoles' visual appearance…

PS1 vs. N64 comparison



Title card for this comparison video by VCDECIDE between a Playstation and Nintendo 64 version of a game, showing how texture filtering choice can significantly affect the overall image.

PS1 vs. N64 comparison

Okay, that one was pretty bad for the N64. Here's a more favorable one:

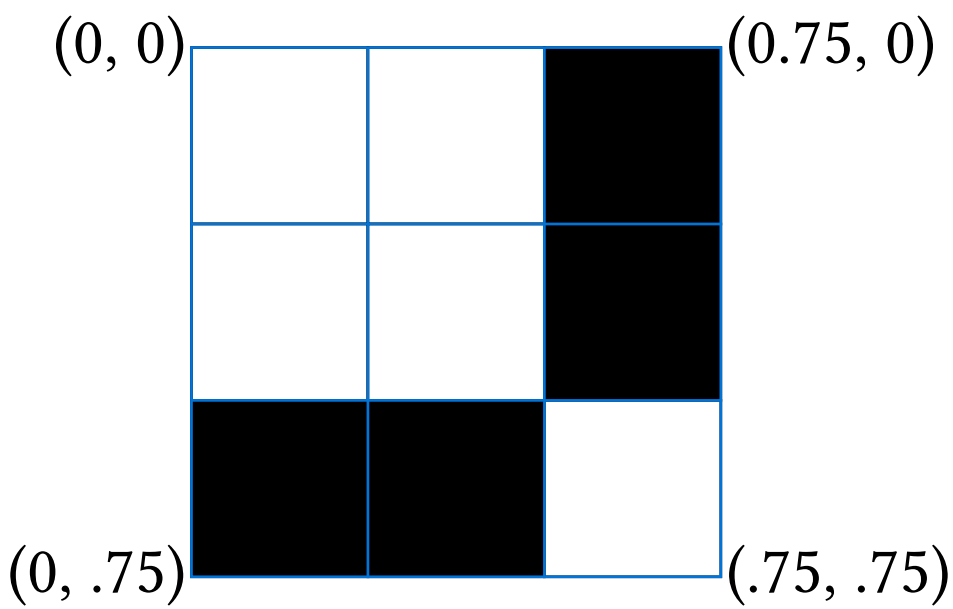


Screenshot from this comparison video by Group M Pro, which also shows how colored lighting has a major effect on graphics.

Questions?

Mapping that texture

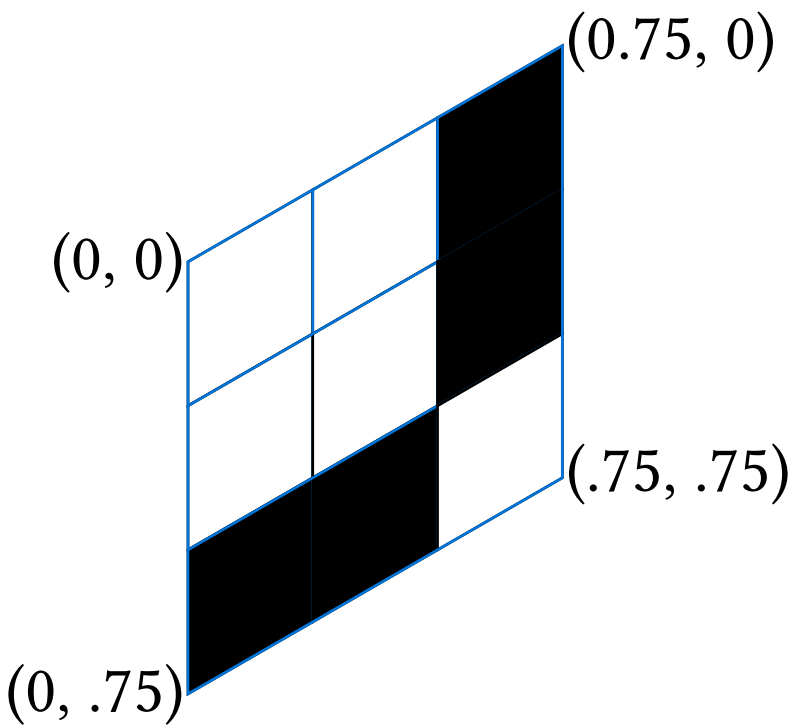
We've already seen how to map textures that aren't distorted. E.g.,:



This is what we get if we use different UV coordinates.

Rotating the texture

But what if we want to rotate the polygon it's mapped onto?



Rotating the texture (2)

Remember that every pixel in that quad is going to have its fragment shader be executed.

The fragment shader will call `textureSample(tex, samp, uv)`.

`tex` and `samp` are constants. But `uv` is an attribute, which means that it is interpolated.

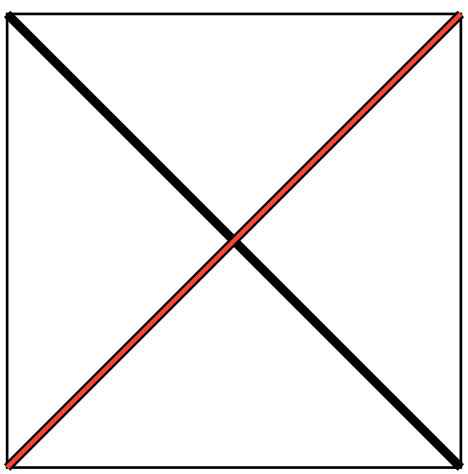
There are actually 3 different ways that it can be interpolated:

- `flat`: the values of the attribute for the first vertex are copied for all the fragments of that triangle.
- `linear`: linear interpolation
- `perspective` (default): perspective-correct interpolation

Linear interpolation:

Flat interpolation is fairly boring, but linear interpolation isn't. It's **almost** what we want, but there's a reason it's not the default.

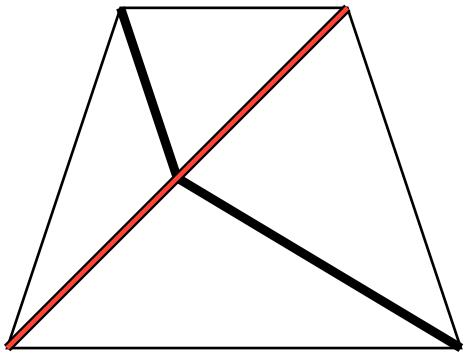
Suppose we want to draw this X-texture to a quad:



The red line is the edge between 2 triangles.

Linear interpolation (2):

Now, suppose we want to rotate it a bit:



Notice that the X is now warped.

For each pixel, we sample a UV coordinate by linearly interpolating between the XYZ coordiantes of each vertex.

Notice that the off-edge lines still touch at the “center” of the edge, but the two triangles don’t agree about how they got there.

Linear interpolation (3):

Imagine the top edge of the quad is 2 distance units away…

For each pixel, we perform the barycentric calculation we talked about waaay earlier in the semester.

Suppose we determine how close we are to the three points of its triangle, and we compute the weighted mean of the UV coordinates for that fragment.

This *does not give a correct result* with textures (or colors, or anything on the surface of the triangle and not at the points).

[Why?]

Linear interpolation (4)

Fundamentally the issue is that correcting for distance puts us in a reciprocal coordinate space.

Remember the w-divide: we put the distance from the camera of each point in the w coordinate.

This makes it so that *the center of the line between two points is not actually where the midpoint would be.*

That's why the perpendicular line stops being perpendicular. The further away we get from the edges of the triangle, the more difference there is between the perspective-correct point and the linearly interpolated texture sample.

Linear interpolation (5)

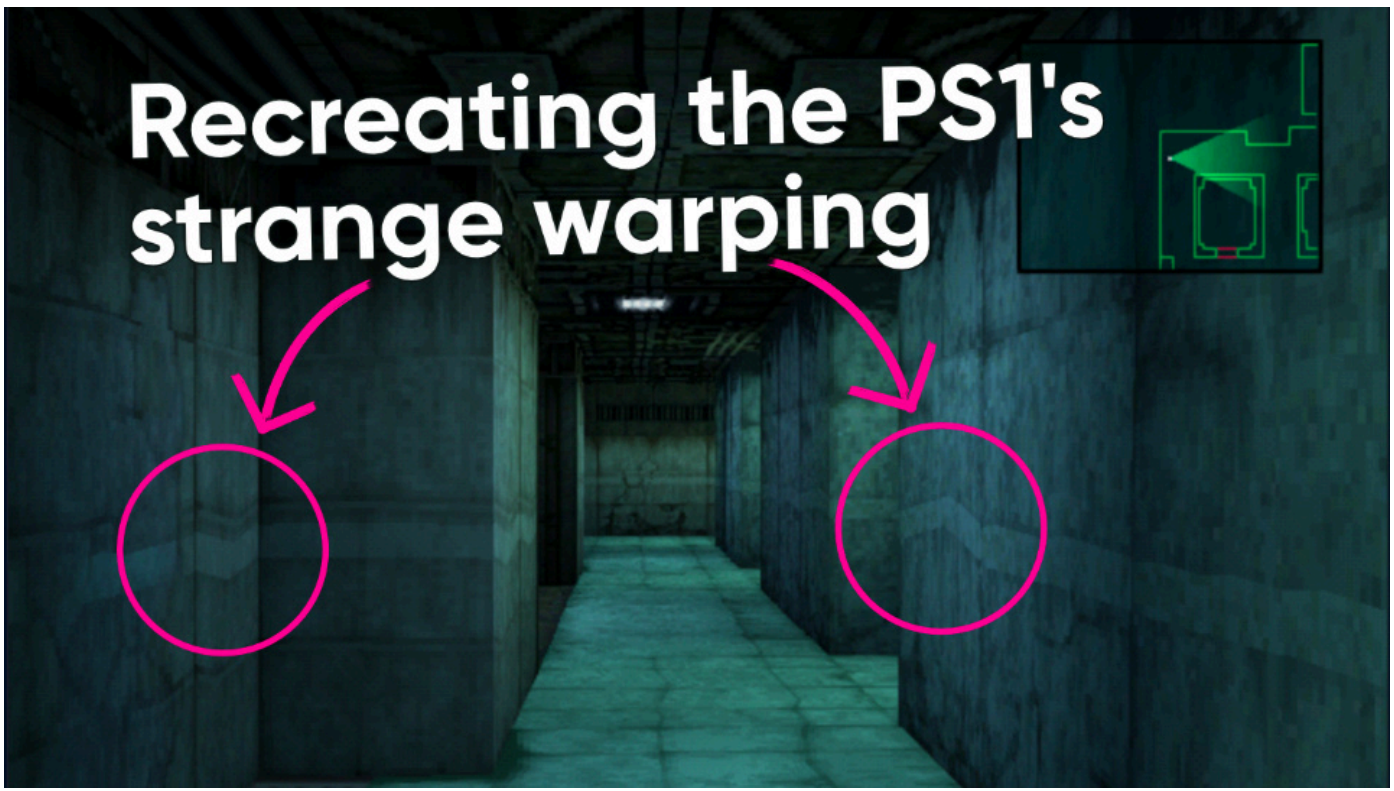
Of course, an X shape is a worst-case scenario for linear interpolation, and it's actually cheaper than the perspective correct option.

In fact, this was the way some early realtime 3D hardware handled textures. It was famously how the Playstation interpolated textures.

Linear interpolation is also called "affine texture mapping".

It led to the famous "PS1 texture warping" ...

Linear interpolation example



Title image from a blog article about recreating the PS1 warping effect using OpenGL

Linear interpolation fixes

It's possible to mitigate the issue with linear interpolation. Notice that only the line that was *not on* the triangle edge was wrong. If you split quads into smaller quads, more of the texture coordinates are close to an edge, making the problem less noticeable.

In fact, this is exactly what Sony recommended developers do, according to this excellent article on making PS1-style graphics.

However, a better solution is to properly account for perspective when *sampling* the texture.

Fun fact

The Nintendo 64 did perspective-correct rendering, allowing level designers to use giant polygons without subdividing them if they wanted.

This would typically result in blurry textures, but they would be perspective-correct blurry textures.

This is one reason why textures often look completely broken with N64 games are ported to the PS1 without adjusting the meshes.

Questions?

Fixing It

Okay, I have a question: how do we fix this? If we can't linearly interpolate over the triangle, then what does the GPU actually do, and why have we never seen that PS1 texture warping (unless some of you deliberately enabled `'linear'` mode to experiment)?

The GPU does **perspective-correct** interpolation, but how?

Well, it has to do with that tricky w-divide again…

Perspective-correct interpolation

The fundamental problem with linear interpolation is that the “depth” dimension does not scale linearly with respect to the X and Y dimensions.

Remember the relationship: `x' = x / d`, `y' = y / d`.

That’s a reciprocal function, not a linear one.

When the pixel in question is on a vertex, it ends up getting the correct value (because there is no interpolation), but otherwise, it’s wrong.

Therefore, we need a different coordinate space in order to interpolate.

Perspective-correct interpolation (2)

Let's say our goal is to compute the correct UV values for each pixel.

Instead of just interpolating U and V directly, we first interpolate `u / w` and `v / w`...

So suppose we have two vertices in which the close one is `w = 1`, and the far one is `w = 10`. And suppose the u coordinate of the close one is `u = 0.5`, and the u coordinate of the far one is `u = 0.75`.

Instead of linearly interpolating between `0.5` and `0.75` as we go along the triangle, we linearly interpolate between `0.5 / 1` and `0.75 / 10`...

Perspective-correct interpolation (3)

Of course, `0.75 / 10 == 0.075`, which is not the correct U coordinate, so we also have to undo this transformation.

We do that by dividing by `w` each time we need to sample the interpolated attribute (i.e., at each fragment).

So let's say we're halfway between the first vertex and the second one:

- Linearly interpolate between `0.5` and `0.075`: `50% * 0.5 + 50% * 0.075 = 0.2875`
- Linearly interpolate between `1 / 1` and `1 / 10` (i.e., the reciprocal of w): `50% * 1 + 50% * 1/10 == .55`
- [Then what?]

Perspective-correct interpolation (4)

Once we have both the interpolated `u/w` and `1/w` values, we divide `u/w` by the interpolated `1/w`.

In our case: `0.2875 / .55 =~ .523`

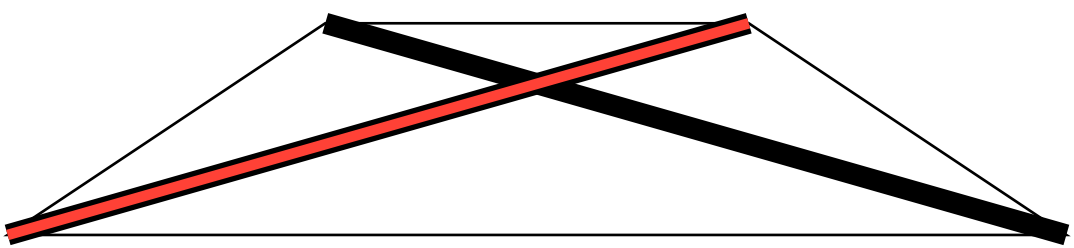
Note: that *is* between `0.5` and `0.75`, but it's much closer to `0.5`.

Let's gain more confidence that it will interpolate by picking a value closer to the distant vertex: `w = 9.9`:

- `1% * 0.5 + 99% * 0.075 == 0.07925`
- `1% * 1 + 99% * 1/10 == 0.109`
- `0.07925 / 0.109 =~ 0.727` . Very close!

What it does

Fundamentally, when we do this kind of interpolation, we place the "center" of the texture appropriately farther away than just the "center" of the x.



Notice how the "center" of the X is proportionately closer to the top edge. That means we interpolate the closer part of the texture much more slowly (causing it to cover more area). This is what we observed with the `w = 5` being much closer to the close value.

Summary of perspective correct sampling:

So, when you use the default `perspective` mode for attribute interpolation in the fragment shader, here is what you're actually doing:

- After the vertex shader produces the `position` value, we use the `w` coordinate to divide *all* the attributes we selected `perspective` for (potentially including UV, normal, and color)
- For each attribute `a`, interpolate `a / w` instead of `a` by itself.
- In addition, interpolate `1 / w`, which has the same nonlinear relationship with the screen-space fragments.
- Finally, divide `a / w` by `1 / w` to get the final attribute.
- If this is a UV coordinate, we can use this new interpolated value to lookup the correct sample of the texture.

Questions?

What even *is* a texture?

If the transformation stuff wasn't weird enough, this is really going to be uncomfortable.

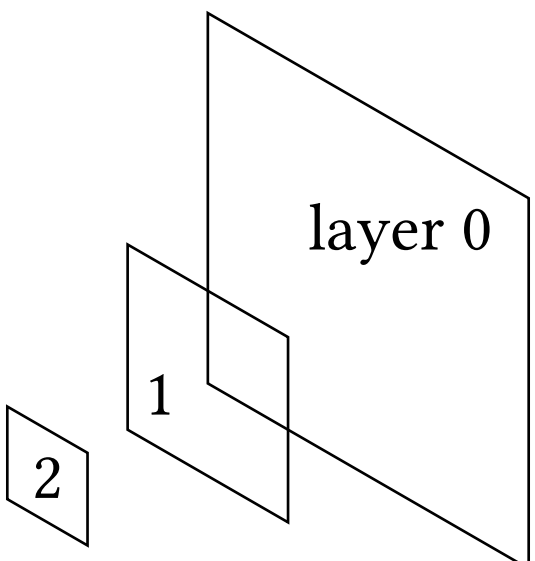
Make sure you're sitting down, this is a lot to take in…

What if I told you… a 2D texture is not actually just a 2D image…



What even *is* a texture? (2)

It's usually a *pyramid* of 2D images, each one a quarter the size (i.e., half the side lengths) of the one before:



This pattern usually continues until the last layer has a single pixel.

What even *is* a texture? (3)

The base layer of the texture, layer 0, is the original image. Typically an uncompressed version of whatever was stored in the texture file. Pretend it's a 1024-by-1024 texture.

Layer 1 is a 512-by-512 version of the same texture. It's just lower resolution. Each pixel is the average between 4 pixels in the larger layer.

Layer 2 is 256-by-256.

And so on…

[For an `n-by-n` texture, how do we calculate how many layers we need to get to 1 pixel?]

But *why*?

Why on earth would we do that?

Well, we didn't always. Before the Nintendo 64, a lot of consumer 3D hardware did not work this way.

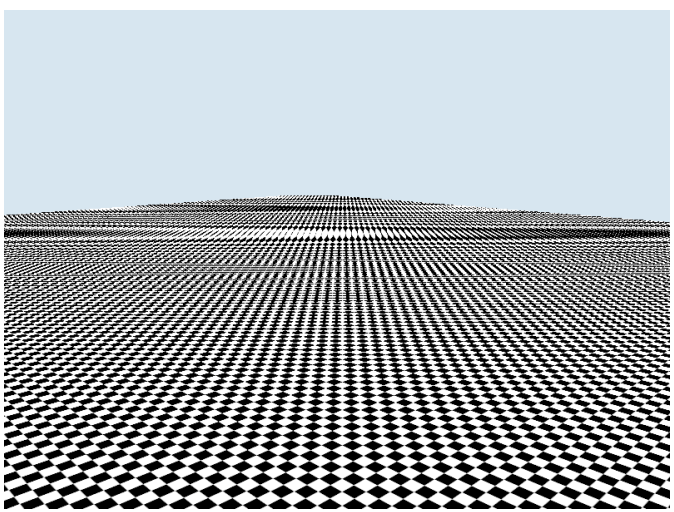
Let's look at why that was a problem…

A simple scene

Let's look at `sample15` together. Ignore the sphere for now.

Instead, focus on the horrible shimmering.

This is called **texture aliasing**.



What causes that?

That looks nothing like a checkerboard in the distance, why?

The issue is that when we get far enough away, and when the surface is parallel to the look direction, we end up sampling the texture *very far apart*.

That is, from one pixel to another, in the vertical direction, we end up sampling a pixel that is very far away from the previous pixel.

As a result, the samples end up being practically random. And as we move, they shimmer and make weird wave patterns.

Texture aliasing

This is actually the same phenomenon as when we see a video of a car's wheel rotating at too low a frame rate and it looks like it's rotating backwards.

It's a term from signal processing called **aliasing**.

When we sample a texture at too low a rate, we end up losing the detail the image was intended to convey, and we get a spurious signal instead.

Ideally, we could just have so many pixels that we could sample it as finely as our eyes do. Unfortunately this is impracticle.

How do we fix it?

If you've been playing around with the sample, you've probably seen some of those checkboxes. The first one holds the solution.

It's a technique called **MIP Mapping**.

MIP is an acronym for *multum in parvo*, a Latin acronym that the creator, Lance Williams, intended to mean "many things in a small space"[1]. It was named and described in this paper.

MIP mapping is an approach to texture sampling that partially solves the "oversampling" problem.

[1] According to Kagi's online translator, a closer translation is "much in little"

MIP mapping

MIP mapping works like this:

- We pre-average together the pixels in the image to create a lower-resolution, bilinear-filtered image in which each texel is perfectly in the middle of the texels of the parent image.
- We compute how much a change in *pixel* results in a change in *texel*.
- That is, if going one pixel right or down results in jumping 2 texels, we use the second mip level.
- If we're jumping 4 texels, we use the third, and so on.
- This averages out the samples, and prevents flickering. Instead of a bunch of wildly different colors, we end up sampling a stretched, blended, texel.

Min filters versus mag filters

Why doesn't bilinear filtering help with this?

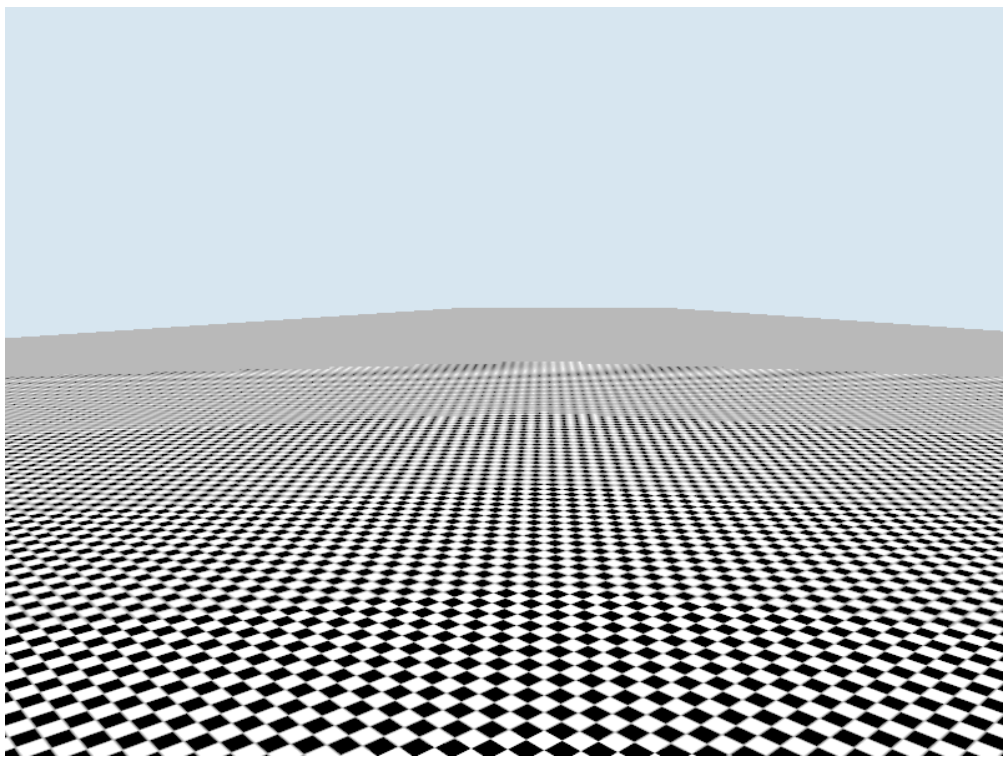
Because bilinear filtering only helps when the image is *blown up*. When there are multiple pixels for every texel, we can smoothly average between texels. That is what bilinear filtering is for.

But MIP mapping is for when the image is *too small*. When we jump too many texels for every pixel, that's when we get the shimmering.

The same approach, a 2-way average, ends up being useful, but we average the texels together to make fewer texels rather than averaging a point between texels.

Results of using MIP mapping

Anyway, we no longer have the shimmering and extreme waviness...



Results of using MIP mapping

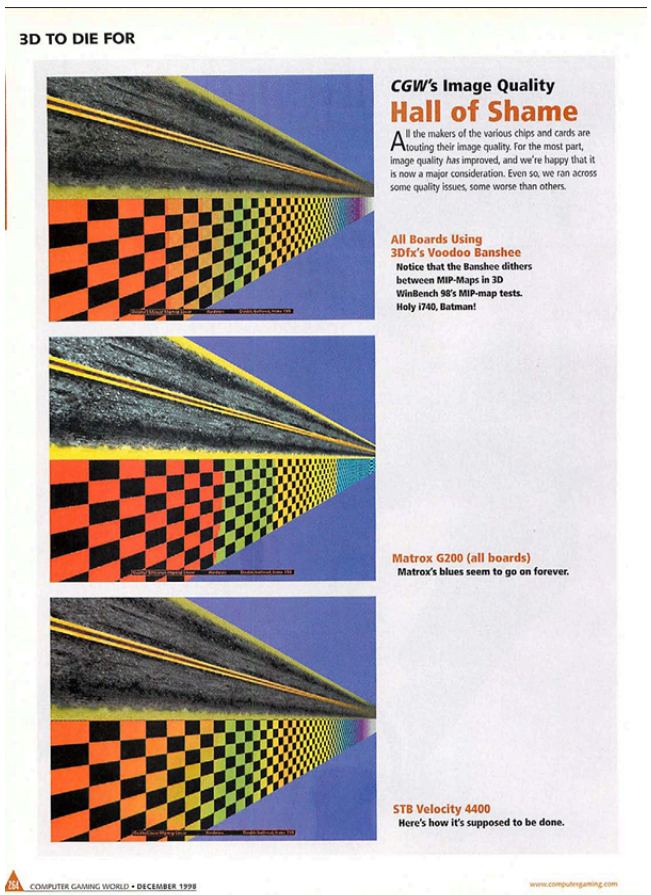
Unfortunately, we now have two, different negative things:

1. It's basically a gray blob towards the horizon.
2. You can see the precise moment it becomes a gray blob.

In fact, there are lots of "boundary lines" visible in the checkered floor.

Those are the different **MIP levels**. We can actually see the point where we cross over from one to the other.

Vendor comparisons



This used to be an area in which GPUs differentiated themselves.

Nowadays, you should see the same results in pretty much any modern GPU.

Source: Computer Gaming World, December 1998 issue, Ziff-Davis Publishing

Trilinear filtering

You might think this is something we could subdivide our way out of, like, just make more mip levels?

In theory we could, but there's a way better solution that doesn't require more texture memory. It's called **trilinear filtering**.

Recall that bilinear filtering meant averaging the four texels around a point to smoothly interpolate its color.

Trilinear filtering is where we also do that *between the two closest MIP levels*.

Trilinear filtering (2)

So now we're doing this:

- Sample the two closest MIP levels (so if our "step" is 2.5 texels per pixel, that puts us between MIP level 1 and 2 (and closer to 1)
- Sample *eight* texels instead of four. The four we would have sampled from MIP level 1, and the corresponding 4 in MIP level 2.
- Perform bilinear filtering for both of these MIP levels individually.
- Compute the weighted mean, but with the weight being the fractional part of the base-2 logarithm of the step: `lg(2.5) =~ 1.32`[1], so 1 - 32% = 68% is the weight we use for the MIP level 1 bilinear sample, and 32% is the weight we use for MIP level 2.

[1] in computer science we often like to use `lg` to mean base-2 logarithm because we use it more than base-10 or e

Basic mip level selection

The reason we use the fractional part of the base-2 logarithm is that the logarithm tells us which MIP level to use.

For example, if we have a step of 1 texel per fragment, great, that's what the texture was designed for. Use MIP level `lg(1) = 0`.

If we have a step of 2 texels per pixel, use MIP level `lg(2) = 1`.

If we have a step of 4 texels per pixel, use MIP level `lg(4) = 2`.

But what if we have 2.5? Then we weighted mean the mip levels, not the number of texels. We want to select an imaginary MIP level between 1 and 2 based on the step size we're given.

Basic mip level selection

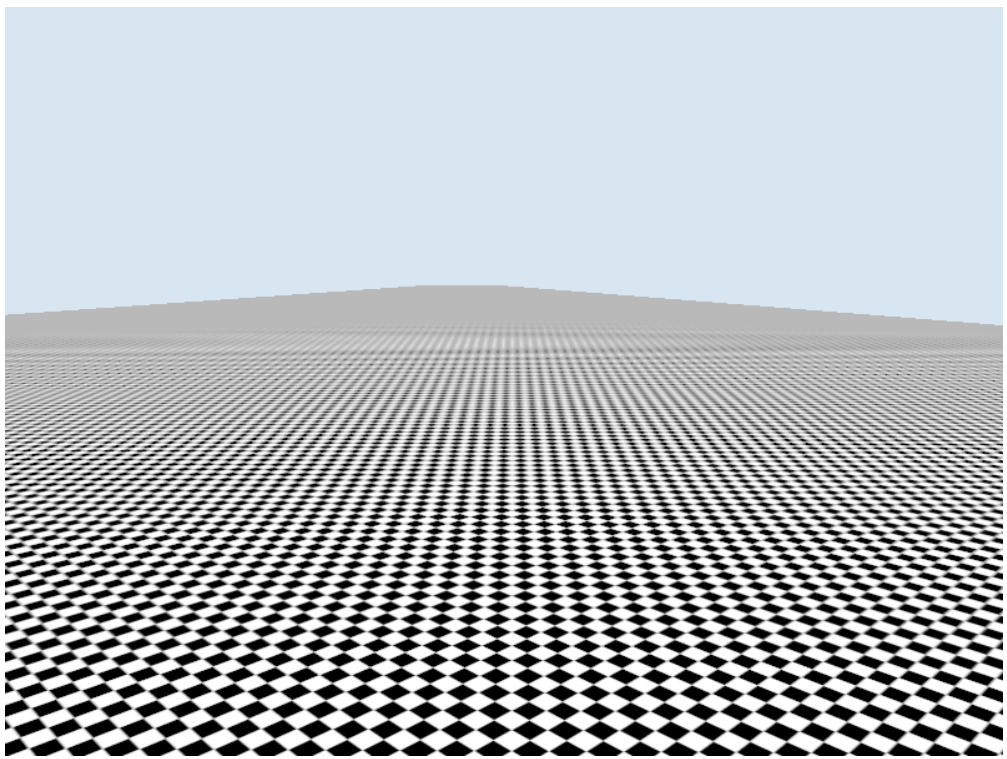
So if our texels per fragment is 2.5, we want level `lg(2.5) =~ 1.32` .

That's what trilinear filtering amounts to: it allows us to select "fractional" MIP levels.

As a result, we no longer have those horizontal bars as we sample textures that recede into the distance.

Let's see how it looks…

Trilinearly filtered sample



Ah, no obvious horizontal lines showing where one MIP-level begins and the other ends! (still a gray blob though)

questions?

Generating MIP levels

Okay, but how do we do this for real?

First of all, we create textures the same way as before:

Only this time, when creating the texture, we specify more MIP levels:

```
const tex = device.createTexture({
    format: 'rgba8unorm-srgb',
    size: [dim, dim, 1],
    usage: GPUTextureUsage.COPY_DST |
        GPUTextureUsage.TEXTURE_BINDING |
        GPUTextureUsage.RENDER_ATTACHMENT,
    mipLevelCount: nMipLevels, // use a positive integer here
});
```

Reminder about MIP mapping as a technique

Remember that the issue that MIP mapping is supposed to solve is oversampling: we're using a large texture but sampling points very far apart on the texture, but close together on the screen.

So once we start skipping texels, we want to create new, bigger texels that average groups of texels in the original image. This way, instead of randomly selecting a black or a white texel, we always select a gray one.

The easiest way to do this is to make a new texture that is half as wide and half as tall. So every texel in the new, smaller texture averages together 4 texels in the original image.

How many MIP levels?

Okay but we had to put an actual number on how many MIP levels we wanted as soon as we create the texture. How many do we use?

Let's see if we can figure out the relationship for a 128-by-128 texture:

1. If we're stepping by 1 texel per fragment, we use the base texture.
2. At 2 texels per fragment, we use MIP level 1, which averages together every cluster of 4 texels. Therefore, it is 64-by-64
3. At 4 texels per fragment, we use MIP level 2 which is 32-by-32.
4. At 8 texels per fragment: MIP level 3, 16-by-16
5. At 16 texels per fragment: MIP level 4, 8-by-8
6. At 32: MIP level 5, 4-by-4
7. At 64: MIP level 6, 2-by-2
8. At 128: MIP level 7, 1-by-1

How many MIP levels? (2)

Once we hit a 1-by-1 texture, there's no point in adding more MIP levels because we always sample a constant color anyway. That's the point where our chess-board tile turns permanently gray.

But what's the relationship? Every time we go up a MIP level, the dimensions of our texture half, so [how many MIP levels do we need?]

How many MIP levels? (3)

Answer: it's logarithmic.

The number of MIP levels is `lg(side_length) + 1`

I'm assuming that the texture is square, so `side_length` is either width or height, and they're both the same.

We almost always use square textures (it was required way back in the day), but we don't have to. If you use a non-square texture, just pick the larger dimension to determine how many mip-maps you need.[1]

---

[1] We also almost always use power-of-2 textures, but if you don't, just round the number of MIP levels down. (The last level will still be 1-by-1, no need for another "fractional" level after it).

Generating MIP maps

Okay, so we've determined *how many* MIP levels we need. But, how do we actually create them. Do we call a `generateMipMaps` function or something?

Sadly, that function used to exist (in OpenGL), but in WebGPU it no longer does. Maybe to keep the driver small or something.

So now we have to make the MIP maps ourselves. This isn't hard, but it's pretty much always done the same way so it's surprising to me that they got rid of the convenient function.

It probably causes a lot of duplicated code.

Generating MIP maps (2)

To generate MIP maps, we basically need to do this:

- Start by loading the full-size texture as MIP level 0
- Then, we chunk MIP level 0 into groups of 2-by-2 texels, and compute their average, to produce MIP level 1
- Then we do the same thing to MIP level 1 to produce MIP level 2
- etc.

We could manually use for-loops to average the texels together. That works and you're allowed to do it. But this is a trivially parallelizable operation that is kind of perfect for the GPU to do.

Generating MIP maps (3)

Here's the typical way to do it:

- Create a sampler that uses bilinear filtering.
- Create a shader that has pre-programmed XY and UV coordinates to draw a quad.
- Texture the quad with the base MIP level.
- Have the destination surface be the *next* MIP level.
- Draw the quad
- Repeat, but the base MIP level will be the one you just rendered, and the destination surface will be the level after that.
- Repeat until every MIP level is filled.

Why does this generate a MIP map?

It seems like all we're doing is drawing a quad over and over again…

The reason it works is that each time we draw a quad, it's smaller.

We use a 64-by-64 texture to draw to the 32-by-32 MIP level.

As a result, the center of the top left texel in the 32-by-32 texture is in between the top left 4 texels in the 64-by-64 texture.

Bilinear filtering averages them together. So the top left texel in the small level is the average of the top left four texels in the bigger level, and so on for each distinct chunk of 4 texels in the base image.

MIP generation shader

See `sample15` for the following:

- A shader to generate MIP levels (it's just drawing a textured quad with hardcoded XY and UV coordinates)
- A function, `createMipmapPipeline`, that generates the pipeline that uses this shader.
- A function, `genMips`, which creates and executes the command encoder that actually binds each MIP level and draws the quads.

MIP generation shader (2)

We select a specific MIP level by creating a texture view:

```
{binding: 0, resource: tex.createView({
    baseMipLevel: layer - 1,
    mipLevelCount: 1,
})},
```

`mipLevelCount` means “only use 1 MIP level, don’t try to go to a higher level when you see that we’re drawing to a smaller quad”.

```
view: tex.createView({
    baseMipLevel: layer, // this is where we draw to
    mipLevelCount: 1,
})
```

Overhead

Those MIP levels are stored inside the texture, in the different layers of the “pyramid”.

This makes the texture bigger, but you probably don’t want to skip on mipmapping to save texture memory.

Remember that each level is a quarter the size of the previous one. Total overhead will be below 50%.

Driver behavior

Once there are more than 1 mipmap level available in the texture view we've bound, the driver will automatically use them.

It will assume that level 1 is to be used when stepping 2 texels per fragment, level 2 is to be used when stepping 4 texels per fragment, etc.

If there aren't enough levels, it will use the highest one.

So, to "turn off" MIP mapping, just use a texture with only level 0 (which is what we were doing so far).

MIP mapping is not all or nothing. You can freely mix MIPped textures and non-MIPped textures (as `sample15` does with the earth model).

Sampling

We enable bilinear filtering like this when creating a sampler:

```
const sampler = device.createSampler({
    magFilter: 'linear',
    minFilter: 'linear',
    mipmapFilter: 'nearest', // don't blend between levels
})
```

`magFilter` is used when the texels per fragment is less than 1, `maxFilter` is used when it is greater than one. We typically use bilinear filtering for both to avoid pixellation.

Sampling (2)

To enable trilinear filtering, we also want to use linear filtering between MIP levels:

```
magFilter: 'linear',
minFilter: 'linear',
mipmapFilter: 'linear', // blend between levels
```

That's it. Swapping out a sampler that has `linear` as the mipmap filter will eleminate the horizontal bands between MIP levels.

Questions?

But I have glossed over something

Remember how I've been talking about the "texels per fragment" rate?

As in, once the texture is farther away, moving one fragment over causes a jump of 2 texels, or 3, or 4, etc.

But there are two bits that I've glossed over:

1. How does the GPU know how many texels we're sampling per fragment? What if the shader is sampling random fragments?
2. Why are we assuming the step rate is the same in both dimensions? What if texels per fragment is 1 in the X direction, but 10 in the Y direction (maybe the quad is rotated to be nearly horizontal)

Let's start with 1

How does the GPU know the texel rate?

In the shader, we use `textureSample` to sample the texture (straightforwardly)

`textureSample` takes a texture, a sampler, and some coordinates.

Normally those coordinates are interpolated UV coordinates which come from the vertex shader.

However, they don't have to. They can literally be anything at all. You can compute the coordinates however you want.

So how does the GPU estimate how many texels are being skipped when it could be different for every fragment?

The GPU does something weird

So far, I have given you a mental model of how the fragment shader works that is reasonable, but not detailed enough to understand MIP selection.

Basically, I've said that first, the rasterizer computes which fragments are part of a triangle, and then the fragment shader runs on each one.

There's actually one more step that happens before the fragment shader: the fragments are *chunked* into 2-by-2 fragment squares.

That is, fragments never run bythemselves. They always run alongside 3 other fragments that are adjacent to them.

The GPU does something weird (2)

Okay…why? Why would the fragments all be grouped into squares?

Because we can use the other fragments in the same square to determine attributes' rate of change along both dimensions.

xy=12, 2 uv=.4,.05	xy=13, 2 uv=.5,.05
xy=12, 3 uv=.4,.1	xy=13, 3 uv=.5,.15

Here's a 2-by-2 quad of fragments. We've computed their screen X and Y coordinates, as well as the final interpolated attributes (after the reciprocals are interpolated)

The Jacobian

This matrix allows us to compute a *Jacobian*, which is a matrix of partial derivatives. If you haven't had multivariate calculus, we can compute a rate of change in only one dimension: this is a partial derivative.

We can compute `du/dx` to determine how much the `u` coordinate changes for one fragment to the right.

In this case, a `1` increase in `x` results in a `.1` increase in `u`, so `du/dx = 0.1`for the top row.

How many texels is that? It depends on the texture. If the texture is 128-by-128, that would be 12.8 texels per pixel horizontally, which would be between levels 3 and 4

The Jacobian (2)

We were able to compute this because for each fragment in the quad, there is a fragment vertically adjacent and horizontally adjacent.

These might not perfectly agree: you might get a different value for the bottom row than the top row, or the left column vs. the right column.

Your driver can decide what to do here (and some APIs expose this choice). For example, it can average the values together in the 2-by-2 block, or simply allow them to differ.

The point is, because we group fragments with their neighbors, we can estimate derivatives now, and use those derivatives to select MIP levels.

What if the dimensions don't agree?

So, here's a problem: sometimes our dimensions don't agree.

In this previous example, the top left had `du/dx = 12.8` and `dv/dx = 0` texels per pixel in X, for a total magnitude 12.8 texels per pixel of UV.[1]

But in Y, in the top left, we had `dv/dy=0.05` and `dv/dx=0`, which is half that rate: `6.4` texels per pixel.

So which one does the video card use?

[1]The change in V doesn't have to be zero, both U and V can change with X, I'm just keeping the math simple.

What if the dimensions don't agree? (2)

By default, when ordinary mip-mapping is used, the video card has to use the *larger* of the two rates of change to avoid shimmering.

That is, it computes this maximum:

$$\rho = \max\left(\sqrt{\left(\frac{\partial u}{\partial x}\right)^2 + \left(\frac{\partial v}{\partial x}\right)^2}, \sqrt{\left(\frac{\partial u}{\partial y}\right)^2 + \left(\frac{\partial v}{\partial y}\right)^2}\right)$$

Then, ρ (pronounced "rho") is used to select the MIP level.

Basically, those two square roots compute the total texel-per-fragment magnitude for both X and Y. The larger of the two is used.

That’s why it turns gray

So, this explains why, in the distance, our checkerboard turns into a gray blob, even with Trilinear filtering.

When the surface is very parallel to the camera, these two rates of change will be very different. For every change in X, we might only be advancing by 1 texel, but for a change in Y, we might be advancing by 16.

As a result, we select a much higher MIP level than we need. This causes the texture to become “gray” much sooner and more noticeably.

Questions?

Summary so far

We started with the observation that just sampling textures led to a really annoying shimmering that we wanted to get rid of.

We solved this by storing “pre-averaged” versions of textures which allowed sampling to be more predictable. Instead of wildly different colors, we would sample a blended color.

However, it wasn’t obvious how to know *which* pre-averaged version to sample…

Summary so far (2)

Therefore, we broke fragments into 2-by-2 squares and used them to compute the rates of change

However, this led to obvious banding where we went from one texture to another

We solved this by averaging between the textures, the same way we average between texels. This was **trilinear filtering**.

But because we selected the MIP level based on highest rate of change, it still turned gray toward the horizon.

Fixing the premature MIP selection

Trilinear filtering was the best we could do on a lot of 90′s hardware. The Nintendo 64 had it built-in. However, shortly after, video cards started to support a technique that improved it further.

The key is to recognize that standard MIP mapping techniques assume a property called **isotropy**, which comes from Greek for “equal turning”.

That is, these techniques assume that we perform the same sampling regardless of which way the texture if facing.

This is not actually true, however, and is the reason why we’re sometimes too aggressive in choosing a MIP level.

Anisotropic filtering

There is a special texture filtering mode we can enable in our samplers by setting the `maxAnisotropy` field.

This is an integer, usually in the range `1` to `16`.

Anisotropic sampling is when we recognize that our texels-per-fragment rate can be different for changes in x versus y.

We treat our partial derivatives as the axes of an ellipse. The smaller change is the minor axis, and the bigger change is the major axis.

Anisotropic filtering (2)

We use the minor axis to select the MIP level. If the smaller rate of change is, say, 5 texels per pixel, then we're going to assume a MIP level of about 2.32.

However, the major axis tells us what direction most of the change is happening in. We will sample the MIP level multiple times along a line centered at the actual computed texture coordinate.

The number of samples is determined by the ratio between the major axis and the minor axis.

Specifically: `min(ceil(MajorLength/MinorLength), maxAnisotropy)`

Anistropic filtering (3)

So if the minor axis is 5 texels per pixel, and the major axis is 25 texels per pixel, we will sample the texture along 5 points of a line that cuts through the texture through the interpolated UV in the direction of the major axis.

The specific points are implementation dependent: the simplest thing to do is choose 5 equally spaced points.

Then, we average the samples together.

`maxAnisotropy` ensures that we don't get stuck needing to sample hundreds of points. The most common maximum is 16.

What does this do?

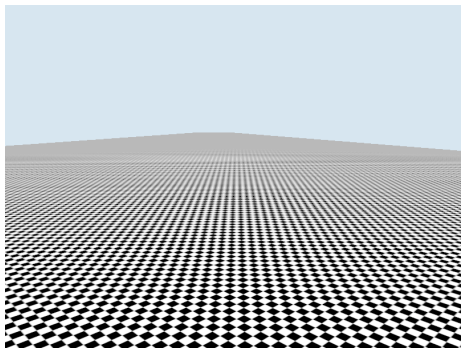
This change allows us to use a lower MIP level: the minimum instead of the maximum.

We still end up sampling multiple times, which can have the effect of averaging together colors, but because we're sampling in a line, if the texels agree along that line, we won't end up with a gray blob.

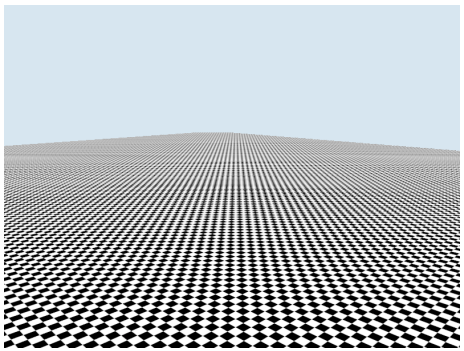
This technique substantially improves the appearance of textures on triangles that are parallel to the viewing direction (i.e., not head on).

Those are the triangles where we would be sampling too high of a MIP level.

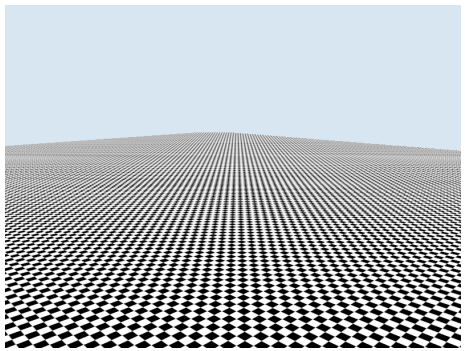
Anistropic filtering improvement



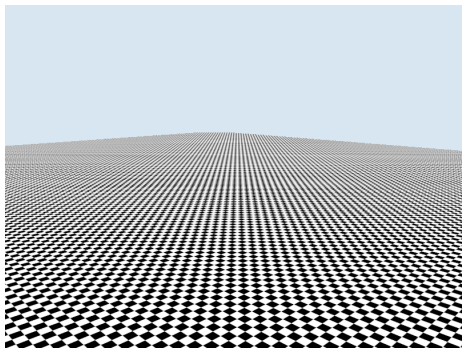
Trilinear only



4x anisotropic



8x anisotropic



16x anisotropic

Notice that you can see further tiles with more anisotropic filtering.

Without it, the horizon appears entirely gray.

Questions?

What if there aren't even 4 fragments?

Okay, so we're using the rate of change texels-per-fragment to determine which mip level(s) and how many samples to use.

Problem: what if it's undefined? Like, the triangle is so small there's only one fragment.

There's a solution, but again, it's weird. We create fake fragments.

The GPU will extend the triangle until it has at least a 2-by-2 quad.

Helper invocations

The fragment executions are called **helper invokations**.

These fragments will be off the face. The fragment shader will still run, but only so that it can compute the texel rate using whatever interpolated data is there. Then it throws the result away.

It also happens when the triangle is large, but one of the 2-by-2 quads happens to fall partially outside of it.

If you have loads of tiny triangles, this could be a significant cost, so it's worth knowing about. In practice, it's not too much overhead.

What if we change the UVs?

What if we do this?

```
textureSample(tex, samp, 2 * uv);
```

With that, we're doubling the sample rate. It's 2-times the texels per fragment.

It turns out, it still works. The GPU will select the correct MIP level.

The reason: it's actually not using the UV coordinate passed to the fragment shader. It's using the UV coordinates passed to `textureSample`. Which raises more questions…

What if we don't sample the texture?

This is a weird one, but it happens. What if our sampling is conditional?

Like this:

```
var color: vec4f;
if something {
  color = textureSample(tex, samp, uv);
} else {
  color = vec4f(1.0, 0.0, 0.0, 1.0);
}
```

What does the GPU use for the UV coordinates when computing the rates of change if you don't call `textureSample`?

You have to sample the texture

It turns out, that code is an error. It won't compile. WGSL requires that `textureSample` be called in **uniform control flow**. That means, if you call it conditionally, it has to be able to prove that the same `textureSample` is called for each fragment in the same draw call.

The actual rules for proving this are complicated. Typically, the best idea is to just always call `textureSample` and refactor. The previous code could be written like this:

```
var color = textureSample(tex, samp, uv);
if something { color = vec4f(1.0, 0.0, 0.0, 1.0); }
```

You have to sample the texture (2)

To be clear, this requirement is per *call site*. Basically, every time you call `textureSample`, it must be from a branch that is uniform. A branch is uniform if:

- it's the main brainch
- it's predicated on a uniform expression (i.e., a boolean that depends only on values that are themselves uniform)

There are more rules for dealing with early returns.

Since this is kind of complicated, it's probably better if you just put your texture samples in the main flow of control.

Ways around it

The issue is that `textureSample` requires the GPU to estimate the texel-fragment derivatives.

However, if you happen to know what the derivative is, either because it's constant or because you have some lookup table, you can manually pass it.

`textureSampleGrad` allows you to provide your own gradient, and `textureSampleLevel` allows you to choose the MIP level. These functions do not have the uniformity requirement.

Here is a list of texture functions for more information.

Is ray tracing looking more appealing?

Man, drawing textured triangles is really hard. Maybe it's not worth it...

Hey what about that ray tracing stuff? Where we simulate light bouncing around. I wonder if that will be practical soon with cheaper cards...

Is ray tracing looking more appealing? (2)

Guess what: you would (practically) still need mipmapping[1].

The fundamental problem is oversampling rather than rasterization.

The techniques are different: we might need to use the chain rule to calculate the change in UV coordinates per XY, or to calculate the "width" of a ray to estimate it.

However, the fundamental calculations in computer graphics: texture sampling, lighting, view transformation, etc., are not that different between rasterization and raytracing.

[1] technicaly you can also just sample tons of times, which many offline raytracers do, but it's not practical for real time.

Questions?

More techniques

Since this lecture isn't already long enough, let's learn a few more techniques.

- Multi-texturing
- Normal mapping
- Specular mapping

The first is easy…

Multi-texturing

So far, we've mainly had only one texture.

Technically, if you did the environment mapping bonus challenge, you've already seen that you can use more than one.

Remember that the fragment shader's main purpose is to return a color.

We can get that from a texture, we can compute it some other way, or we can average a bunch of textures together.

Why multi-texturing?

The first consumer GPU to support multi-texturing was the Nintendo 64's display processor (the *reality display processor* or *RDP*)[1]

It's kind of surprising to me that it was so rare. It allows a bunch of very straightforwardly useful artistic choices, like the ability to paint roads or dirt on things.

See this deep dive for some examples from the game Banjo Kazooie. Look at the farm screenshot for an example of texture blending.

[1] My source is that I made an LLM try to find counter examples. Apparently it took 2 more years for this technology to be found in consumer GPUs like the Voodoo2.

Multi-texturing (2)

Multi-texturing can be used for **detail textures**.

The idea was, you'd have one base texture that's intended to be spread out over a wide area (like grass), and then you'd have a repeating texture with really big UVs.

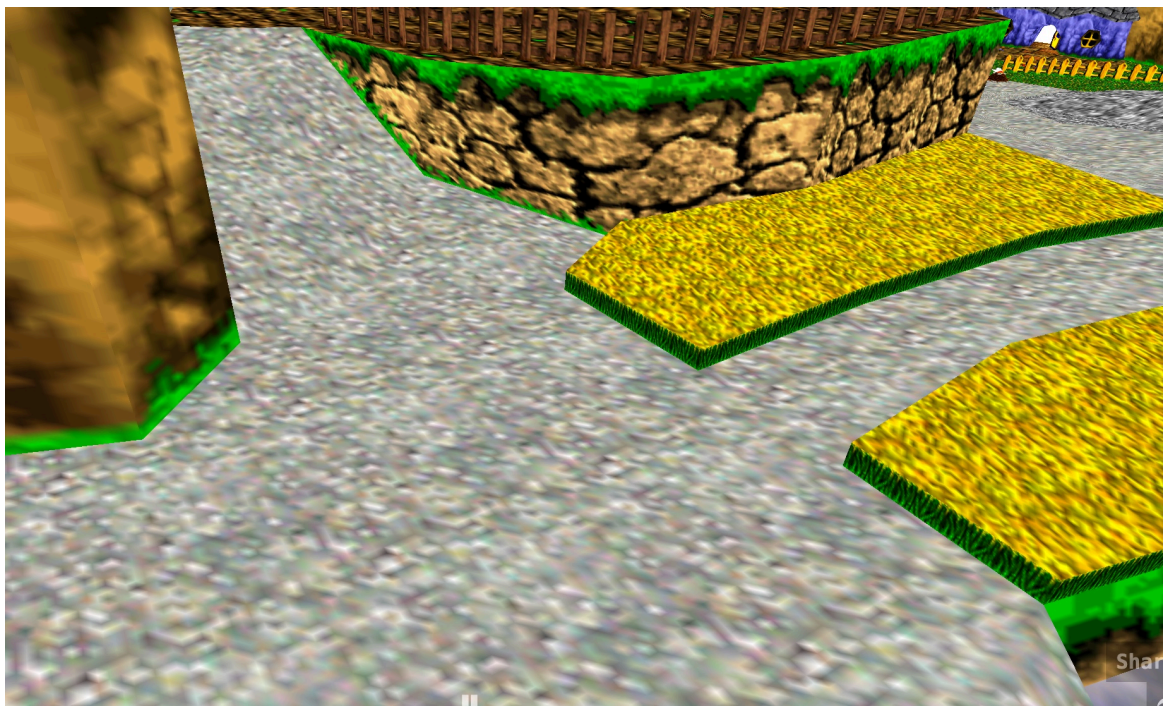
For example, suppose we have a large quad with a UV range of 0 to 1.

However, we also have a detail texture with a UV range of 0 to 10.

The detail texture will repeat 10 times as it is drawn over the quad.

Here's an example…

Multi-texturing (3)



The pathway is a gray blobby texture. However, there is random noise used as a detail texture to give it a gravelly appearance. The noise texture can have bigger UVs to repeat for more detail.

Multi-texturing (4)

The easiest way to get started is just to use two textures with the same UVs. You can blend between textures this way.

However, you can also bind a separate set of UV coordinates as another attribute. So in the same way that position, normal, uv, and color can be attributes, you can also have multiple sets of UV coordinates.

This lets you, e.g., have one texture repeat more quickly than another.

`sample15` shows an example where the clouds are a separate texture that are blended with the earth. I computed the cloud UVs from the normal.

Questions?

Textures that aren't textures

So far, we've been using textures in the way they were intended. As images that get mapped onto triangle faces.

Here's the thing though: textures are just arrays that can be sampled.

So we can actually use them for all kinds of things. Basically anything that an interpolated array is useful for!

Here's a cool example...

Normal mapping

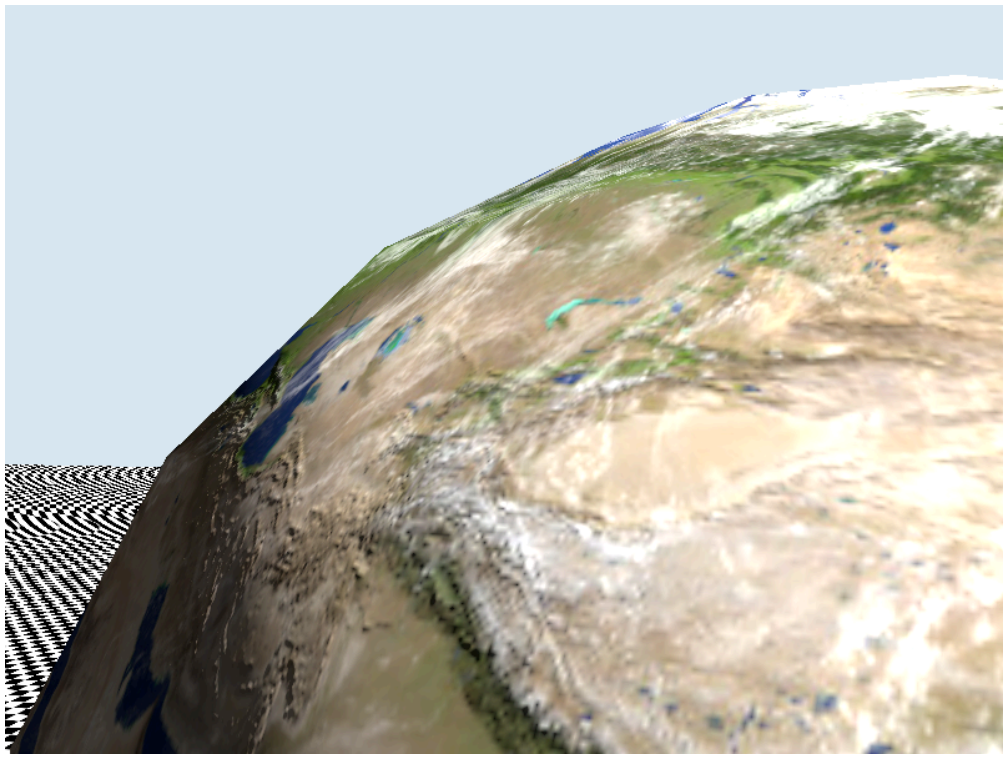
Remember how normals were computed per-vertex?

That was a little weird. We typically associate a normal with a face, rather than a vertex, but we needed them to be accessible to the vertex shader.

Well, it turns out we can store normals in a texture instead. This way they are accessible to the *fragment shader*.

Why is that useful? Because it lets us perform lighting correctly on surfaces that are smaller than a face. Like cracks or small changes in elevation…

Normal mapping example



Notice the shadows cast by the Himalayas and the mountains around Iran. `sample15` uses normal mapping for the globe texture. This lighting changes at the globe rotates.

How normal mapping works

So how can we store normals in a texture? Aren't they vectors?

Yes, and textures store vectors! Specifically, an RGB texture stores texel of 3 dimensions.

We can interpret RGB as XYZ, and pretend that each texel describes a direction rather than a color.

Of course, normal images only give us 8 bits per channel, which is kind of rough for storing a direction. However, many popular image formats such as PNG support 16 bits per channel, which is enough given that small bits of imprecision for a single fragment won't be noticeable.

Normal mapping (2)

However, we don't just want to store the exact direction of the normal. If we did that, our normal map would only be useful for one model.

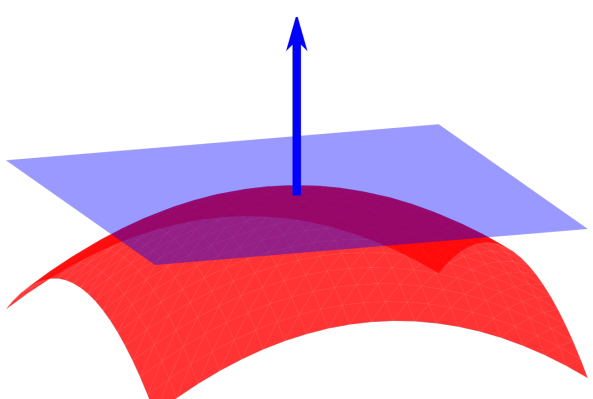
Typically, we distribute normal maps alongside textures, rather than models. They can be made to be flexible for multiple underlying models.

Instead, we typically use a different coordinate system for normal maps called **tangent space**.

Tangent space

The idea is that we store normals such that they are relative to any surface we want.

Typically this means we interpret the normal (0, 1, 0) (i.e., straight up) as being “outward from the surface”



By Patrick87 - Own work based on: Surface normal illustration.png, redone using Matlab R2012a and optimized in Inkscape., Public Domain, Link

Tangent space (2)

That tangent plane in the previous image can be described with two basis vectors. We call them the **tangent vector** and the **bitangent vector**.

Typically, the tangent vector is derived from the cross product of “up” (the 0, 1, 0 vector we treat as coming “out” of the model) and the actual normal coming out of the surface.

This a way of having it be perpendicular to “up” and also perpendicular to the actual normal. So it’s embedded in that tangent plane.

Tangent space (3)

But, we need another basis vector too. So we take another cross product. This one between the normal and the tangent vector. This gives us the counterpart vector to the tangent which is also embedded in the plane.

We call this the **bitangent vector**.

Finally, the third vector we need is one pointing in the same direction as the normal itself....which...is just the normal.

These three vectors will serve as the basis vectors for a new coordinate space, which we'll just call `tbn` for "tangent bitangent normal".

Decoding the normal map

Now that we know what our basis vectors are, we actually know everything we need about how to decode the normals in a normal map.

We want to interpret a pixel in the normal map as a vector, where its `r` component is how much to angle in the tangent direction, its g vector is how much to angle in the bitangent direction, and its b vector is how much to point straight out.

…except for one more wrinkle. RGB images can't have negative color. So we instead treat `0` as `-1`, `128` as `0`, and `255` as `1` and interpolate that way.

Decoding the normal map (2)

So, color $= \frac{\text{normal} + (1\ 1\ 1)}{2}$. This converts the components of the normal from the range $[-1, 1]$ to $[0, 1]$.

We undo this transformation when we sample the texture:

```
var n = textureSample(normalmap, samp, uv).rgb * 2.0 - 1.0;
```

So `n` has our normal as a vector now, taken from the color of the texture.

Our next step is to build that `tbn` matrix...

Decoding the normal map (3)

```
let up = vec3f(0.0, 1.0, 0.0);
let norm_tan = normalize(cross(up, norm));
let norm_bit = cross(norm, norm_tan);
let norm = the actual normal from the vertex shader;
```

Normally, the tangent vector would be stored in the mesh itself, alongside the UV and normal. There is an algorithm for computing it when exporting normal maps called MikkTSpace.

In our case, I'm drawing a sphere, so I know how to compute them: take the cross product with the north pole. But typically `norm_tan` would come interpolated from the vertex shader (and you would normalize).

Decoding the normal map (4)

The purpose of the tangent and bitangent vectors is to serve as basis vectors, along with the normal itself.

This allows us to take the RGB value from the normal map and interpret it as “how much of the tangent to take, how much of the bi-tangent to take, and how much of the original normal to take”.

We want a matrix that takes the shifted color and converts it into the final normal (the one we will use for lighting calculations)…

Decoding the normal map (5)

```
let tbn = mat3x3f(norm_tan, norm_bit, norm);
```

Here, we use the `mat3x3f` constructor to build a 3-by-3 matrix from 3 basis vectors.

One more thing: DirectX and Unreal engine store the Y component upside down. I don't know why. Maybe it's a handedness thing. In my case, the normal map was assuming this convenion:

```
n.y = -n.y; // if normal map uses direct X convension
```

(Unity stores it without flipping y, you just have to know)

Decoding the normal map (6)

Finally, we can use the `tbn` matrix and our sampled unencoded `n` normal from the normal map in order to compute the actual normal we will use for lighting purposes:

```
let light_norm = normalize(tbn * samp_norm);
```

From this point on, we just use ordinary Phong lighting with that value as our normal (for the diffuse and specular).

A typical normal map

This is what a typical normal map looks like:



Majestic.

A typical normal map (2)

Actually, if you zoom way in, and you have good color sensitivity, you can kind of make out the oceans and mountain ranges of planet Earth.

Remember that the way our normal maps are packed, the R channel is the amount the normal is bent left or right away from the mesh normal. The G channel is the amount it's bent up or down. The B channel is the amount that it matches the normal you would expect.

These channels store 0 as 50%. So 0 is −1, and 100% is 1.

Therefore, most of the normal map is going to be very close to 50% R, 50% G, and 100%B. Most normal maps are mostly this shade of Blue.

Questions?

Specular mapping

Remember the Phong model?

Every material had these properties:

- Ambient response
- Diffuse response
- Specular response
- Shininess

Previously, we applied the material to the whole model. So if a sphere were supposed to be made of metal, we could use a metal texture with high specular response and high shininess. If it were meant to be made of wood, we would use high diffuse and low specular and shininess.

Specular mapping (2)

The problem with that approach was that it meant the whole model we drew would have the same material properties.

Now, I'm trying to draw the Earth.

Pop quiz: what is shinier and more specular-y: a lake or a bunch of dirt?

Specular mapping (3)

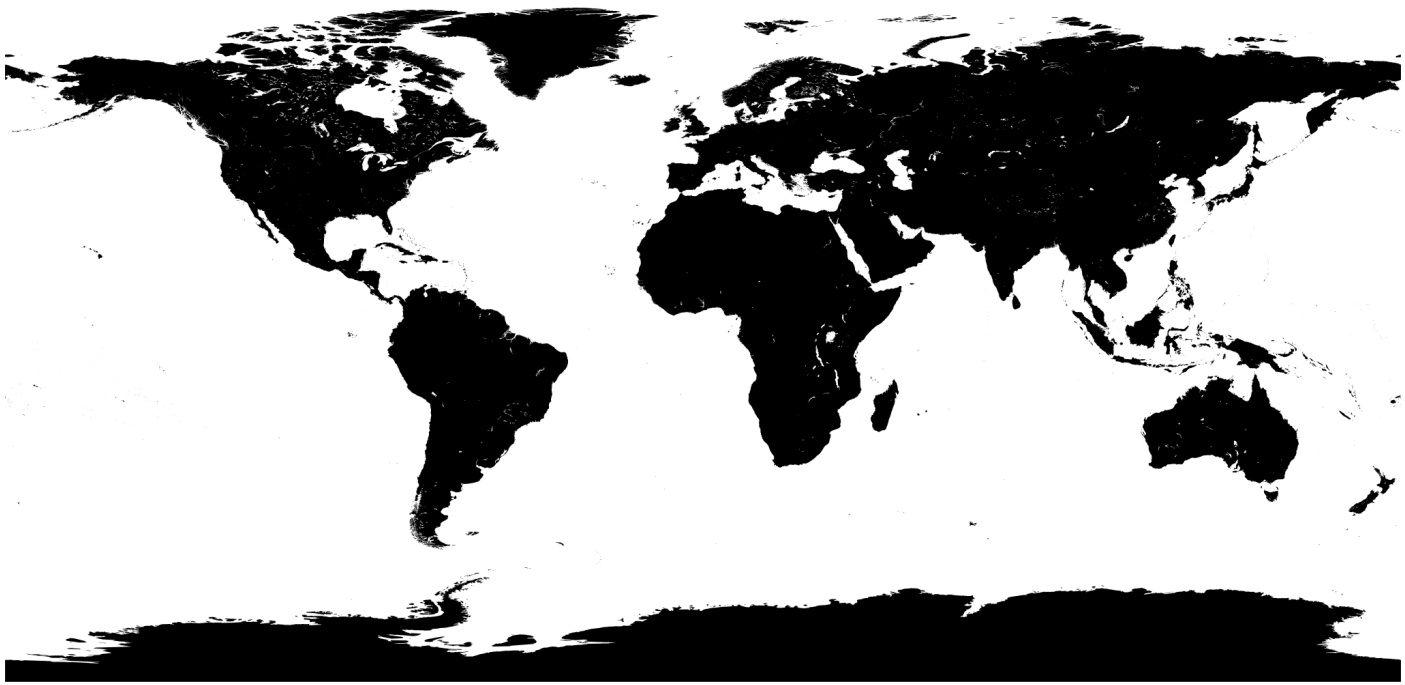
That's right, the lake is shinier.

Unfortunately, using the Phong code from before, if we make the whole earth as shiny as the water, then the land will be shiny too. Or vice versa if we make the land dull.

Advanced textures to the rescue: we can just store the "specular amount" in a texture as a percentage for each texel.

Then, when we compute the brightness in the fragment shater, we can use the correct level of specular for each fragment, after sampling that texture.

Example specular map



The light areas have full specular. The dark areas have limited or no specular component This particular specular map is not very detailed and effectively only has no specular or max specular.

Even more maps

- What if you used a texture where each texel was a height? You could store a whole complex terrain mesh as a single image.
- What if you used a 3D texture as a different color space, so that every time you computed a color in the fragment shader, you looked that color up in the texture to map it to a different color?
- What if you used each channel of the RGBA image as a different material property? So, not just specular, but ambient, diffuse, specular, and shininess?
- What if you want to blend two textures dynamically and unevenly. Like, apply a mud texture to a car paint texture according to a certain pattern, so a 3rd texture gives the amount of mud to add.

Even more maps

These all have names!

- If each texel is a height, it's a height map. If those heights are really tiny and describe the heights of little bumps (which we can also compute normals from) it's a bump map.
- We can use a 3D texture for color grading. It's a color lookup table. If we use it for discrete colors, it's a palette.
- Use each channel as a different value is called "channel packing". We can make a material map with each channel as a different material value.
- Using a texture to blend other textures is a blend map (or alpha map).

Generally they end in "map".

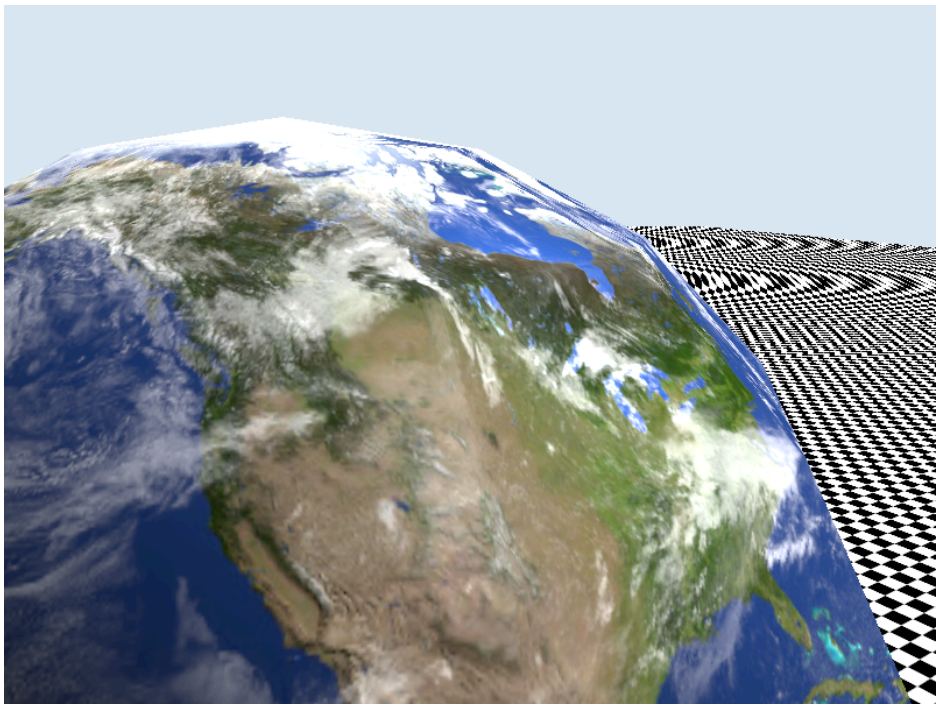
Specular map code

How do we use a specular map? We look up the brightness value and multiply it by the specular component.

How do we get brightness? In this case, the RGB channels all have the same value, so we traditionally use R.

```
let spec_sample = textureSample(specmap, samp, earth_uv).r;
let spec = spec_sample *
  pow(max(dot(r, eye_dir), 0.0), SHININESS);
```

Specular map example screenshot



Specular mapping in action. Notice that there's a bit of shine to the lakes, but the land doesn't have any.

Questions?